

Лабораторная работа №1

Введение в Mininet

Шуваев Сергей Александрович

Содержание

1	Цель работы	4
2	Теоретическое введение	5
3	Выполнение лабораторной работы	6
4	Выводы	15
	Список литературы	16

Список иллюстраций

3.1	Импорт конфигураций	6
3.2	Параметры импорта	7
3.3	Настройка сети	8
3.4	Настройка сети	8
3.5	Запуск mininet	9
3.6	Подключение к mininet через SSH	9
3.7	Просмотр IP-адресов машины	10
3.8	Файл /etc/netplan/01-netcfg.yaml	10
3.9	Обновление Mininet	11
3.10	Номер установленной версии mininet	11
3.11	Настройка соединения X11 для суперпользователя	12
3.12	Работа с Mininet с помощью командной строки	13
3.13	Работа с Mininet с помощью командной строки	13
3.14	Работа с Mininet с помощью командной строки	14
3.15	Проверка связности хостов	14

1 Цель работы

Основной целью работы является развёртывание в системе виртуализации (например, в VirtualBox) mininet, знакомство с основными командами для работы с Mininet через командную строку и через графический интерфейс.

2 Теоретическое введение

Mininet[**mininet?**] – это эмулятор компьютерной сети. Под компьютерной сетью подразумеваются простые компьютеры — хосты, коммутаторы, а так же OpenFlow-контроллеры. С помощью простейшего синтаксиса в примитивном интерпретаторе команд можно разворачивать сети из произвольного количества хостов, коммутаторов в различных топологиях и все это в рамках одной виртуальной машины(ВМ). На всех хостах можно изменять сетевую конфигурацию, пользоваться стандартными утилитами(`ifconfig`, `ping`) и даже получать доступ к терминалу. На коммутаторы можно добавлять различные правила и маршрутизировать трафик.

Mininet создает реалистичную виртуальную сеть, выполняя реальный код ядра, коммутатора и приложения на одной машине (VM, облачной или собственной) за считанные секунды с помощью одной команды `sudo mn`.

3 Выполнение лабораторной работы

Перейдем в репозиторий Mininet, скачаем актуальный релиз ovf-образа виртуальной машины. Запустим систему виртуализации и импортируем файл .ovf и укажем параметры импорта (рис. 3.1;3.2).

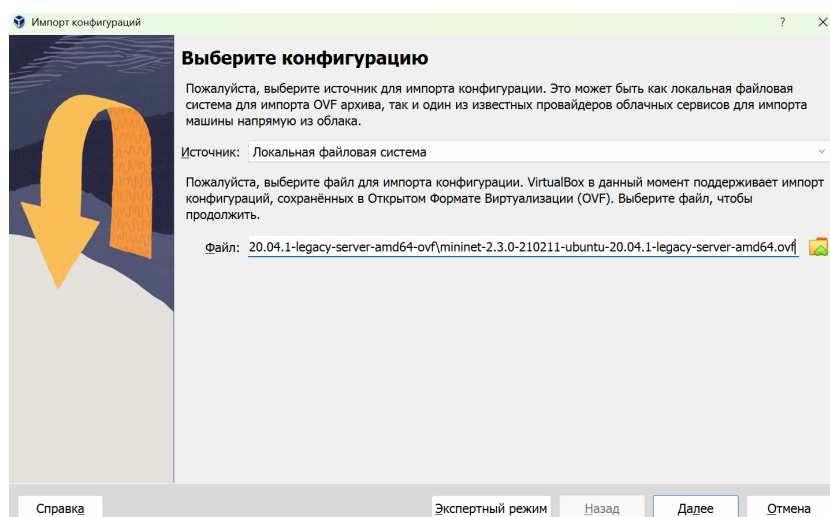


Рис. 3.1: Импорт конфигураций

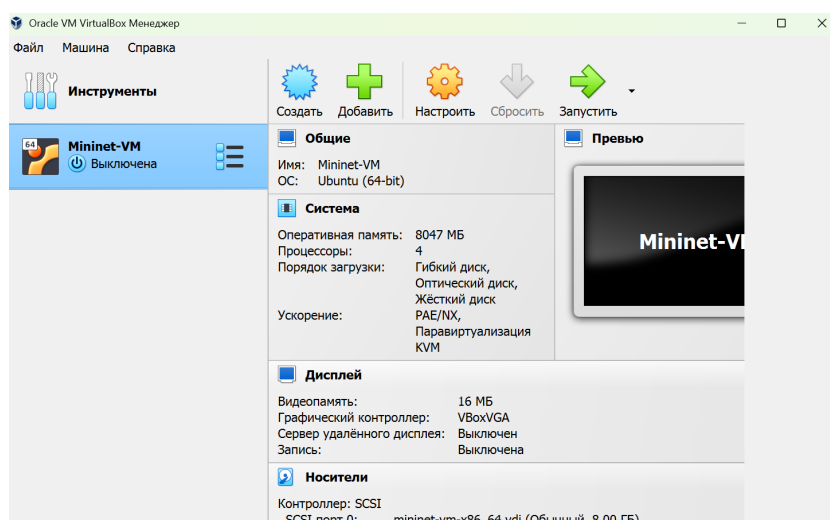


Рис. 3.2: Параметры импорта

Перейдем в настройки системы виртуализации и уточним параметры настройки виртуальной машины. В частности, для VirtualBox выберем импортированную виртуальную машину и перейдите в меню “Машина -> Настроить”. Перейдем к опции «Система». Если внизу этого окна есть сообщение об обнаружении неправильных настроек, то, следуя рекомендациям, внесем исправления (изменим тип графического контроллера на рекомендуемый). В настройках сети первый адаптер должен иметь подключение типа NAT (рис. 3.3). Для второго адаптера укажите тип подключения host-only network adapter (виртуальный адаптер хоста), который в дальнейшем вы будете использовать для входа в образ виртуальной машины (рис. 3.4).

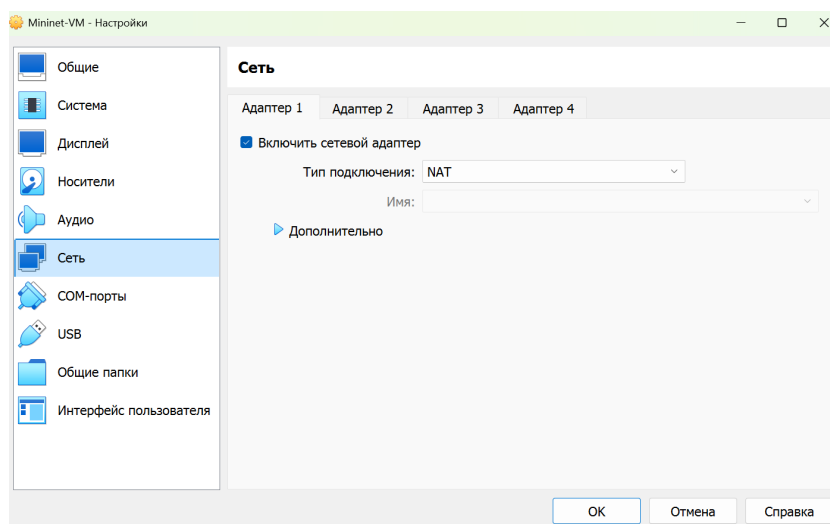


Рис. 3.3: Настройка сети

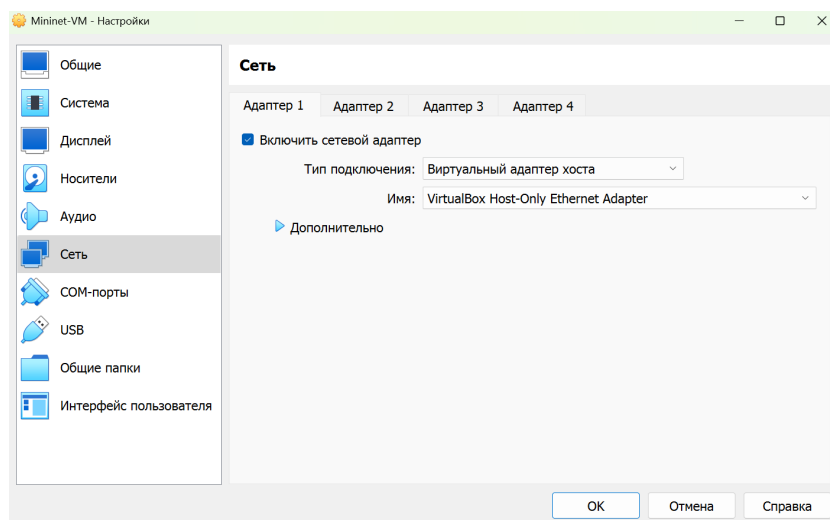


Рис. 3.4: Настройка сети

Запустим виртуальную машину с Mininet. Залогинимся в виртуальную машину:
 - login: mininet - password: mininet
 Посмотрите адрес машины с помощью ifconfig (рис. 3.5).

```
Ubuntu 20.04.1 LTS mininet-vn tty1
mininet-vn login: mininet
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-42-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

Last login: Wed Feb 10 21:03:31 PST 2021 on ttyS0
mininet@mininet-vn:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.56.102 netmask 255.255.255.0 broadcast 192.168.56.255
    ether 08:00:27:03:cc:2d txqueuelen 1000 (Ethernet)
    RX packets 6  bytes 3058 (3.0 KB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 2  bytes 684 (684.0 B)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 128  bytes 9816 (9.8 KB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 128  bytes 9816 (9.8 KB)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

mininet@mininet-vn:~$
```

Рис. 3.5: Запуск mininet

Подключимся к виртуальной машине (из терминала хостовой машины). Настроим ssh-подсоединение по ключу к виртуальной машине. Вновь подключимся к виртуальной машине и убедимся, что подсоединение происходит успешно и без ввода пароля (рис. 3.6).

```
+ xauth list $DISPLAY
+ =====
+ CategoryInfo          : ObjectNotFound: (xauth:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\WINDOWS\system32> xauth add mininet-vn/unix:10
xauth : Имя "xauth" не распознано как имя командлета, функции, файла сценария или выполняемой программы. Проверьте правильность написания имени, а также наличие и правильность пути, после чего повторите попытку.
строка:1 знак:1
+ xauth add mininet-vn/unix:10
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (xauth:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\WINDOWS\system32> ssh mininet@192.168.56.102
mininet@192.168.56.102's password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-42-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Last login: Sat Sep 13 06:47:37 2025 from 192.168.56.1
mininet@mininet-vn:~$ xauth add mininet-vn/unix:10 MIT-MAGIC-COOKIE-1 295acad8e35d17636924c5ab80e8462d
xauth: file /home/mininet/.Xauthority does not exist
mininet@mininet-vn:~$ xauth list $DISPLAY
mininet-vn/unix:10  MIT-MAGIC-COOKIE-1 295acad8e35d17636924c5ab80e8462d
mininet@mininet-vn:~$
```

Рис. 3.6: Подключение к mininet через SSH

После подключения к виртуальной машине mininet посмотрим IP-адреса машины. Активен только внутренний адрес машины вида 192.168.x.y, поэтому активируем второй интерфейс (рис. 3.7).

```

mininet@mininet-vm:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.102 netmask 255.255.255.0 broadcast 192.168.56.255
    ether 08:00:27:03:cc:2d txqueuelen 1000 (Ethernet)
    RX packets 342 bytes 79123 (79.1 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 161 bytes 22674 (22.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2748 bytes 205684 (205.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2748 bytes 205684 (205.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

Рис. 3.7: Просмотр IP-адресов машины

Для удобства дальнейшей работы добавим для mininet указание на использование двух адаптеров при запуске. Для этого требуется перейти в режим суперпользователя и внести изменения в файл /etc/netplan/01-netcfg.yaml виртуальной машины mininet. В результате файл /etc/netplan/01-netcfg.yaml должен иметь следующий вид (рис. 3.8).

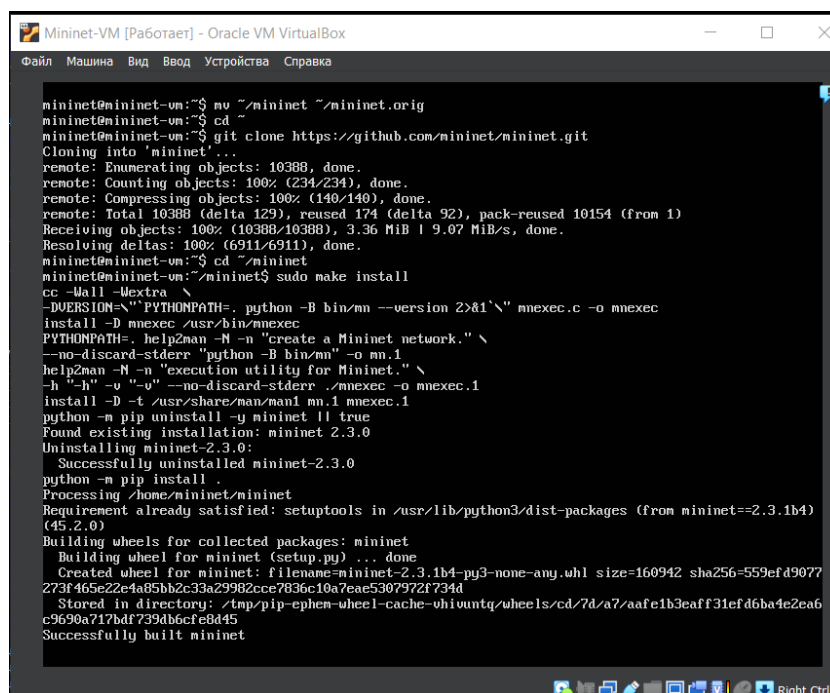
```

mininet@mininet-vm: ~
/etc/netplan/01-netcfg.yaml [-M--] 16 L: [ 1+ 9 10/ 11] *(219 / 220b) 10 0x00A
# This file describes the network interfaces available on your system
# For more information, see netplan(5).
network:
  version: 2
  renderer: networkd
  ethernets:
    eth0:
      dhcp4: yes
    eth1:
      dhcp4: yes

```

Рис. 3.8: Файл /etc/netplan/01-netcfg.yaml

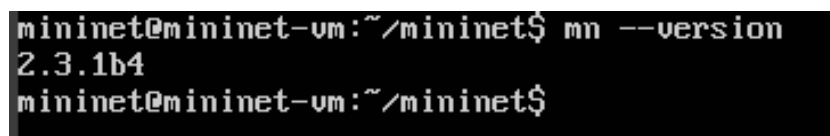
В виртуальной машине mininet переименуем предыдущую установку Mininet. Скачаем новую версию Mininet. Обновим исполняемые файлы (рис. 3.9).



```
mininet@mininet-vm:~$ mv ~/mininet ~/mininet.orig
mininet@mininet-vm:~$ cd ~
mininet@mininet-vm:~$ git clone https://github.com/mininet/mininet.git
Cloning into 'mininet'...
remote: Enumerating objects: 10388, done.
remote: Counting objects: 100% (234/234), done.
remote: Compressing objects: 100% (140/140), done.
remote: Total 10388 (delta 129), reused 174 (delta 92), pack-reused 10154 (from 1)
Receiving objects: 100% (10388/10388), 3.36 MiB | 9.07 MiB/s, done.
Resolving deltas: 100% (6911/6911), done.
mininet@mininet-vm:~$ cd ~/mininet
mininet@mininet-vm:~/mininet$ sudo make install
cc -Wall -Wextra \
-DVERSION="\`PYTHONPATH=. python -B bin/mn --version 2>&1`\`" mnexec.c -o mnexec
install -D mnexec /usr/bin/mnexec
PYTHONPATH=. help2man -N -n "create a Mininet network." \
--no-discard-stderr "python -B bin/mn" -o mn.1
help2man -N -n "execution utility for Mininet." \
-h "-h" -u "-u" --no-discard-stderr ./mnexec -o mnexec.1
install -D -t /usr/share/man/man1 mn.1 mnexec.1
python -m pip uninstall -y mininet || true
Found existing installation: mininet 2.3.0
Uninstalling mininet-2.3.0:
  Successfully uninstalled mininet-2.3.0
python -m pip install .
Processing /home/mininet/mininet
Requirement already satisfied: setuptools in /usr/lib/python3/dist-packages (from mininet==2.3.1b4) (45.2.0)
Building wheels for collected packages: mininet
  Building wheel for mininet (setup.py) ... done
  Created wheel for mininet: filename=mininet-2.3.1b4-py3-none-any.whl size=160942 sha256=559efd9077273f465e22e4a85bb2c33a29982cce7836c10a7eae5307972f734d
  Stored in directory: /tmp/pip-ephem-wheel-cache-vhivuntq/wheels/cd/7d/a7/aafc1b3eaff31efd6ba4e2ea6c9690a717bdf739db6cfe8d45
Successfully built mininet
```

Рис. 3.9: Обновление Mininet

Проверим номер установленной версии mininet (рис. 3.10).



```
mininet@mininet-vm:~/mininet$ mn --version
2.3.1b4
mininet@mininet-vm:~/mininet$
```

Рис. 3.10: Номер установленной версии mininet

При попытке запуска приложения из-под суперпользователя возникает ошибка: X11 connection rejected because of wrong authentication. Ошибка возникает из-за того, что X-соединение выполняется от имени пользователя mininet, а приложение запускается от имени пользователя root с использованием sudo. Для исправления этой ситуации необходимо заполнить файл полномочий /root/.Xauthority, используя утилиту xauth. Скопируем значение куки (MIT magic cookie)¹ пользователя mininet в файл для пользователя root (рис. 3.11).

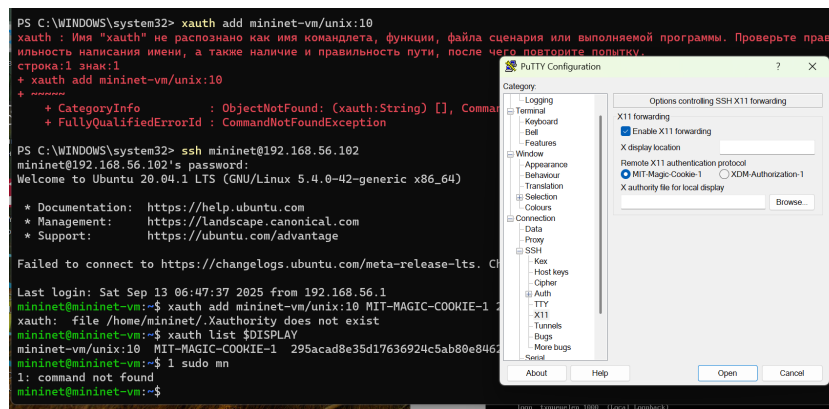


Рис. 3.11: Настройка соединения X11 для суперпользователя

Для запуска минимальной топологии введем в командной строке (рис. 3.12): `sudo mn`. Эта команда запускает Mininet с минимальной топологией, состоящей из коммутатора, подключённого к двум хостам. Для отображения списка команд интерфейса командной строки Mininet и примеров их использования введем команду в интерфейсе командной строки Mininet: `help` Для отображения доступных узлов введем: `nodes` Вывод этой команды показывает, что есть два хоста (хост h1 и хост h2) и коммутатор (s1). Иногда бывает полезно отобразить связи между устройствами в Mininet, чтобы понять топологию. Введем команду `net` в интерфейсе командной строки Mininet, чтобы просмотреть доступные линки: `net` Вывод этой команды показывает: - Хост h1 подключён через свой сетевой интерфейс h1-eth0 к коммутатору на интерфейсе s1-eth1. - Хост h2 подключён через свой сетевой интерфейс h2-eth0 к коммутатору на интерфейсе s1-eth2. - Коммутатор s1: - имеет петлевой интерфейс lo. - подключается к h1-eth0 через интерфейс s1-eth1. - подключается к h2-eth0 через интерфейс s1-eth2.

```

mininet@mininet-vm:~$ sudo dhclient eth1
Cannot find device "eth1"
mininet@mininet-vm:~$ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> nodes
available nodes are:
c0 h1 h2 s1
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0
c0
mininet>

```

Рис. 3.12: Работа с Mininet с помощью командной строки

Mininet позволяет выполнять команды на конкретном устройстве. Чтобы выполнить команду для определенного узла, необходимо сначала указать устройство, а затем команду, например: `h1 ifconfig`

Эта запись выполняет команду `ifconfig` на хосте `h1` и показывает интерфейсы хоста `h1` — хост `h1` имеет интерфейс `h1-eth0`, настроенный с IP-адресом `10.0.0.1`, и другой интерфейс `lo`, настроенный с IP-адресом `127.0.0.1`.

```

mininet> h1 ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.1 netmask 255.0.0.0 broadcast 10.255.255.255
    ether d2:06:bb:63:97:84 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

mininet>

```

Рис. 3.13: Работа с Mininet с помощью командной строки

Посмотрим конфигурацию всех узлов.

```

mininet> h2 ifconfig
h2-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
        inet 10.0.0.2 netmask 255.0.0.0 broadcast 10.255.255.255
        ether 8e:65:d6:ca:d4:38 txqueuelen 1000 (Ethernet)
        RX packets 0 bytes 0 (0.0 B)
        RX errors 0 dropped 0 overruns 0 frame 0
        TX packets 0 bytes 0 (0.0 B)
        TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
        inet 127.0.0.1 netmask 255.0.0.0
        loop txqueuelen 1000 (Local Loopback)
        RX packets 0 bytes 0 (0.0 B)
        RX errors 0 dropped 0 overruns 0 frame 0
        TX packets 0 bytes 0 (0.0 B)
        TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

Рис. 3.14: Работа с Mininet с помощью командной строки

По умолчанию узлам h1 и h2 назначаются IP-адреса 10.0.0.1/8 и 10.0.0.2/8 соответственно. Чтобы проверить связь между ними, используем команду ping. Команда ping работает, отправляя сообщения эхо-запроса протокола управляющих сообщений Интернета (ICMP) на удалённый компьютер и ожидая ответа. Например, команда h1 ping 10.0.0.2 проверяет соединение между хостами h1 и h2.

```

mininet> h1 ping 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.60 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.200 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.030 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.036 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.091 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.035 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.036 ms
^C
--- 10.0.0.2 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6108ms
rtt min/avg/max/mdev = 0.030/0.290/1.603/0.538 ms
mininet> exit

```

Рис. 3.15: Проверка связности хостов

4 Выводы

В результате выполнения данной лабораторной работы я развёрнул mininet в системе виртуализации VirtualBox, а также ознакомился с основными командами для работы с Mininet через командную строку и через графический интерфейс.

Список литературы